

النتيجة:

```
# إلى 5 طباعة الأرقام من 1
count = 1 # متغير التحكم

while count <= 5: # الشرط
    print(f"العدد: {count}") # طباعة العدد الحالي
    count = count + 1 # زيادة متغير التحكم

print("انتهت الحلقة") # رسالة النهاية
```

النتيجة:

```
العدد: 1
العدد: 2
العدد: 3
العدد: 4
العدد: 5
انتهت الحلقة
```

الموضوع السابع | تحديث قيم المتغيرات والحلقات اللانهائية

أهمية تحديث متغير التحكم:

 يجب تغيير قيمة المتغير في كل تكرار وإلا ستحصل على حلقة لانهائية.

مثال صحيح

```
x = 0
while x < 3:
    print(x)
    تحديث مهم # x = x + 1
```

```
(حلقة لانهائية) مثال خاطئ #
# x = 0
# while x < 3:
#     print(x)
#     x! نسيان تحديث #
```

ما هي الحلقة اللانهائية؟

 حلقة لا تتوقف أبداً

 تحدث عندما يبقى الشرط True دائماً

الموضوع السابع | استخدام break و continue

أهمية تحديث متغير التحكم:

```
count = 0
while count < 10:
    زيادة العدد # count += 1

    if count == 5:
        5تخطي العدد # continue

    if count == 8:
        8خروج عند الوصول للعدد # break

    طباعة العدد # print(count)
```

تعليمة break:



تخرج من الحلقة فوراً

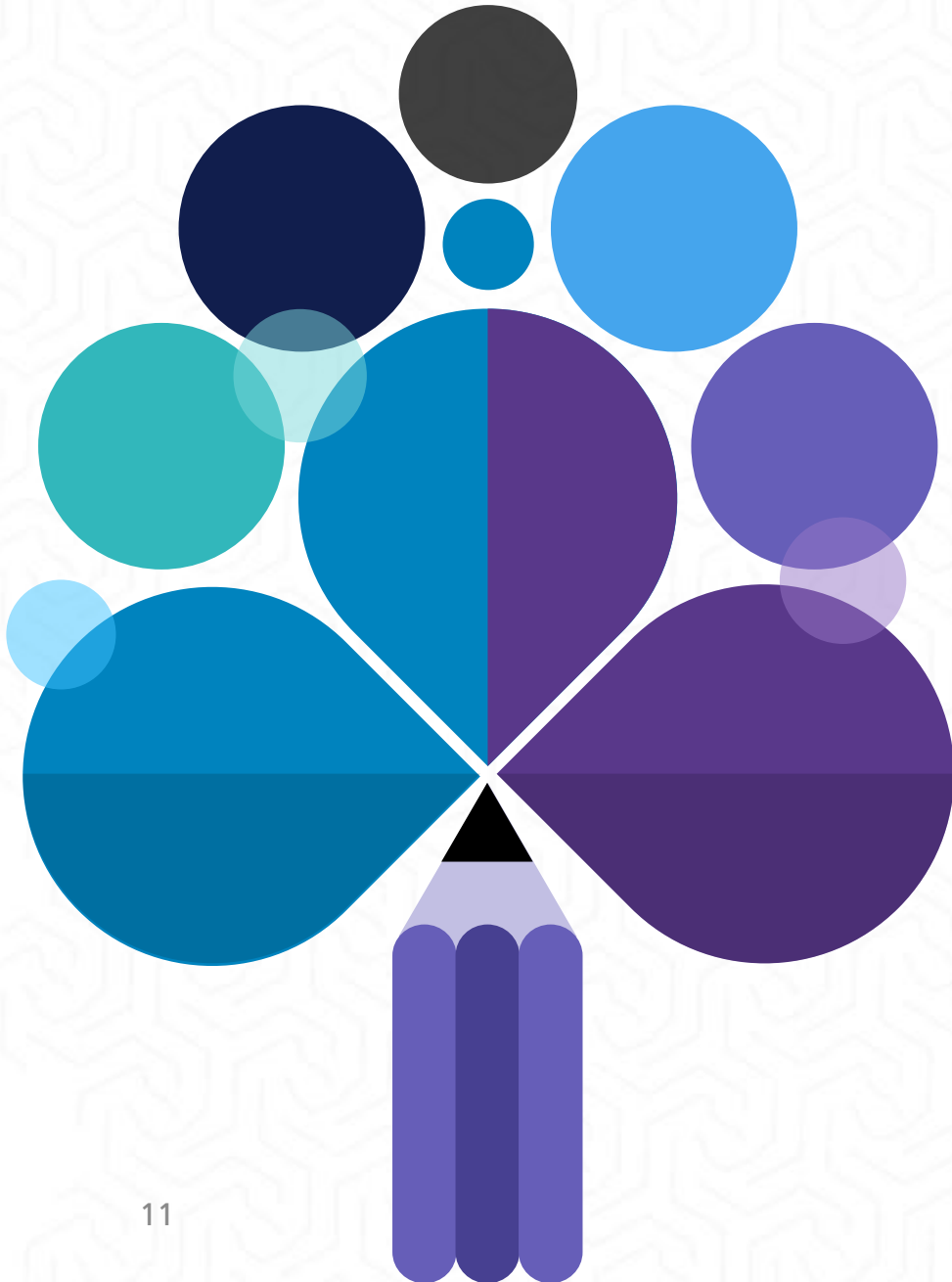
تتجاهل باقي التعليمات في الحلقة.

تعليمة continue:

تتخطى باقي التعليمات في التكرار الحالي

نتنقل للتكرار التالي

الموضوع السابع | مثال عملي - آلة حاسبة بسيطة



```
print("آلة حاسبة بسيطة")  
print("للخروج 'quit' أدخل ")
```

```
while True:  
    # إدخال المعادلة  
    equation = input("مثلاً: 5 + 3: ")
```

```
if equation.lower() == 'quit': # فحص إذا أراد المستخدم الخروج  
    print("وداعاً!")  
    break
```

```
try:  
    result = eval(equation) # حساب النتيجة  
    print(f"النتيجة: {result}")
```

```
except:  
    print("رسالة خطأ #") # خطأ في المعادلة، حاول مرة أخرى
```

الموضوع السابع | مقدمة عن حلقة for

حلقة: for:

تتكرر على عناصر مجموعة (سلسلة، قائمة، إلخ). 

عدد التكرارات محدد مسبقاً. 

أكثر إيجازاً من while في كثير من الحالات. 

الصيغة العامة

for variable in sequence:
كود للتنفيذ



الموضوع السابع | حلقة for مع القوائم

قائمة من الأسماء

```
names = ['عائشة', 'محمد', 'فاطمة', 'أحمد']
```

```
print("أسماء الطلاب:")
```

```
for name in names: # التكرار على كل عنصر في القائمة  
    print(f"- {name}")
```

حساب مجموع الأرقام

```
numbers = [10, 20, 30, 40, 50]  
total = 0
```

```
for number in numbers: # التكرار على كل رقم  
    total += number # إضافة الرقم للمجموع
```

```
print(f"المجموع: {total}") # طباعة النتيجة النهائية
```



الموضوع السابع | حلقة for مع السلاسل النصية

التكرار على أحرف النص

```
text = "Python"
```

```
print("أحرف الكلمة:")
```

```
for char in text: # التكرار على كل حرف
```

```
print(f"الحرف: {char}")
```

عد الأحرف المتحركة

```
vowels = "aeiouAEIOU" # الأحرف المتحركة
```

```
vowel_count = 0
```

```
word = "programming"
```

```
for letter in word: # فحص كل حرف
```

```
if letter in vowels: # إذا كان حرف متحرك
```

```
vowel_count += 1 # زيادة العداد
```

```
print(f"عدد الأحرف المتحركة في '{word}': {vowel_count}")
```

دالة range():

تنتج سلسلة من الأرقام 

مفيدة مع حلقة for للتكرار عدد محدد من المرات 

range(n) - من 0 إلى n-1

```
print("4 إلى 0 الأرقام من:")  
for i in range(5): # 0, 1, 2, 3, 4  
    print(i)
```

range(start, stop) - من start إلى stop-1

```
print("\n5 إلى 1 الأرقام من:")  
for i in range(1, 6): # 1, 2, 3, 4, 5  
    print(i)
```

range(start, stop, step) - بخطوة محددة

```
print("\n10 إلى 0 الأرقام الزوجية من:")  
for i in range(0, 11, 2): # 0, 2, 4, 6, 8, 10  
    print(i)
```

الموضوع السابع | أنماط الحلقات - العد والجمع

حساب عدد العناصر - نمط العد

```
fruits = ['تفاح', 'موز', 'برتقال', 'تفاح', 'عنب', 'تفاح']  
apple_count = 0
```

التكرار على كل فاكهة #

إذا كانت تفاح: #

زيادة العداد # apple_count += 1

```
print(f"عدد التفاحات: {apple_count}")
```

حساب المجموع - نمط الجمع

```
prices = [10.5, 25.0, 15.75, 8.25, 30.0]
```

```
total_price = 0
```

التكرار على كل سعر #

إضافة السعر للمجموع # total_price += price

```
print(f"إجمالي الأسعار: {total_price}")
```



الموضوع السابع | أنماط الحلقات - إيجاد القيم الكبرى والصغرى

```
scores = [85, 92, 78, 96, 88, 91, 83]
```

إيجاد أعلى درجة #

قيمة ابتدائية فارغة # `highest_score = None`

التكرار على كل درجة # `for score in scores:`

`if highest_score is None or score > highest_score:`

تحديث أعلى درجة # `highest_score = score`

`print(f"أعلى درجة: {highest_score}")`

إيجاد أقل درجة #

قيمة ابتدائية فارغة # `lowest_score = None`

التكرار على كل درجة # `for score in scores:`

`if lowest_score is None or score < lowest_score:`

تحديث أقل درجة # `lowest_score = score`

`print(f"أقل درجة: {lowest_score}")`



الموضوع السابع | الحلقات المتداخلة

طباعة جدول الضرب

```
print("جدول الضرب:")  
print("-" * 30)
```

3 إلى 1 الصفوف من

```
for i in range(1, 4): # 1 الأعمدة من 3  
    for j in range(1, 4): # حساب النتيجة  
        result = i * j  
        print(f"{i} × {j} = {result}") # طباعة المعادلة  
    print("-" * 15) # فاصل بين الصفوف
```

البحث في قائمة من القوائم

```
students_grades = [  
    ['أحمد', 85, 90, 78],  
    ['فاطمة', 92, 88, 95],  
    ['محمد', 76, 82, 89]  
]
```

التكرار على كل طالب

```
name = student[0] # اسم الطالب  
total = 0
```

التكرار على درجات الطالب

```
total += grade # جمع الدرجات  
average = total / (len(student) - 1) # حساب المتوسط  
print(f"متوسط{name}: {average:.1f}")
```

الموضوع السابع | التعامل مع القواميس في الحلقات



قاموس بيانات الطلاب

```
student_scores = {  
    'أحمد': 85,  
    'فاطمة': 92,  
    'محمد': 78,  
    'عائشة': 96,  
    'علي': 88  
}
```

التكرار على المفاتيح

```
print("أسماء الطلاب:")  
for student in student_scores: # التكرار على المفاتيح  
    print(f"- {student}")
```

التكرار على القيم

```
print("\nالدرجات:")  
for score in student_scores.values(): # التكرار على القيم  
    print(f"- {score}")
```

التكرار على المفاتيح والقيم معاً

```
print("\nالطلاب ودرجاتهم:")  
for student, score in student_scores.items(): # التكرار على الأزواج  
    print(f"{student}: {score}")
```

الموضوع السابع | مثال عملي - تحليل النصوص

text = "Python is a powerful programming language. Python is easy to learn."

تحويل النص إلى أحرف صغيرة وتقسيمه #

```
words = text.lower().replace('.', '').replace(',', '').split()
```

إحصاء الكلمات #

```
word_count = {}
```

التكرار على كل كلمة #

```
for word in words: # إذا كانت الكلمة موجودة
```

```
    word_count[word] += 1 # زيادة العداد
```

```
else:
```

```
    word_count[word] = 1 # إضافة الكلمة الجديدة
```

عرض النتائج #

```
print("إحصائيات الكلمات:")
```

```
for word, count in word_count.items(): # عرض كل كلمة وعددها
```

```
    print(f"'{word}': {count} مرة")
```

إيجاد الكلمة الأكثر تكراراً #

```
most_common_word = None
```

```
max_count = 0
```

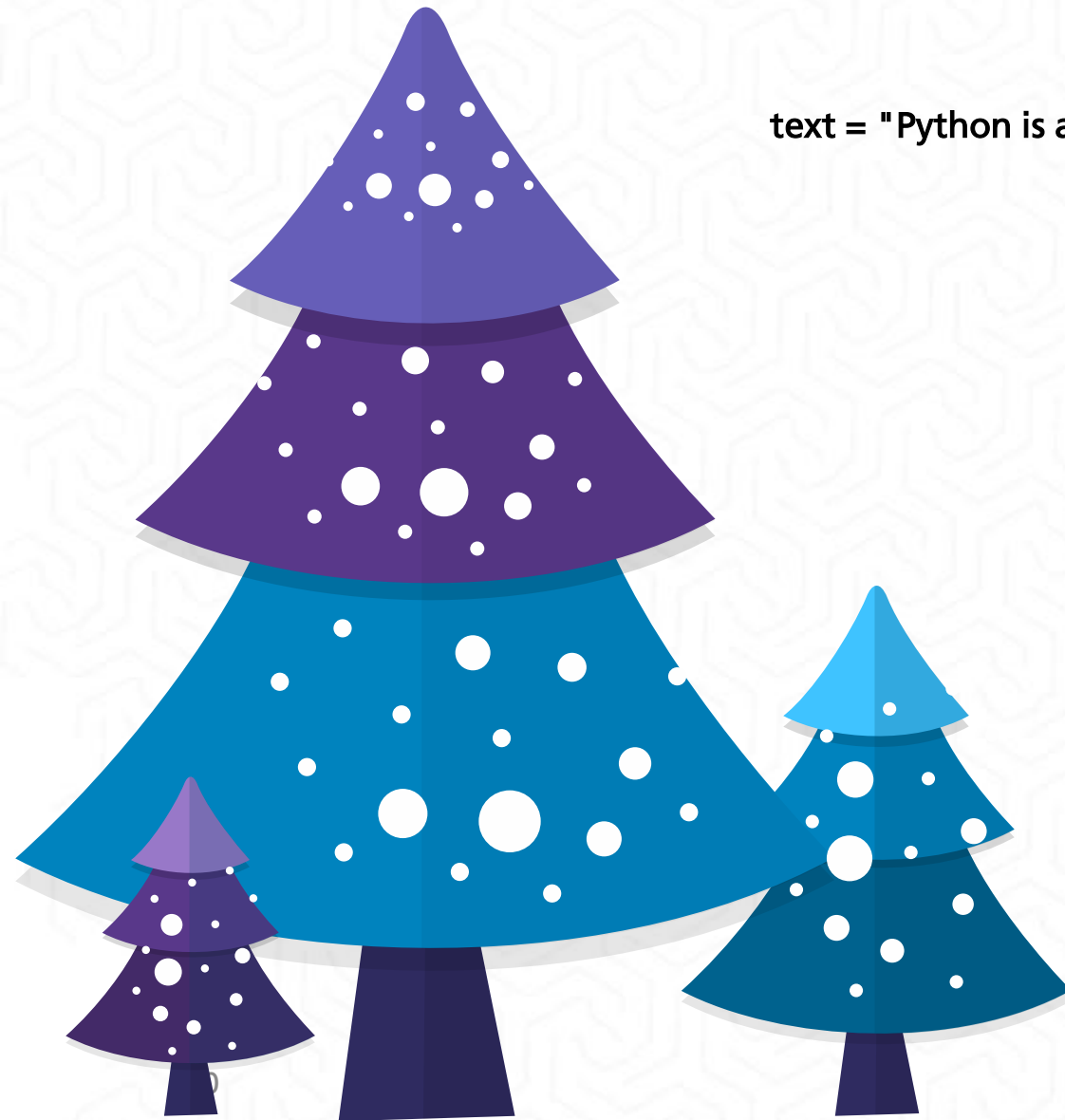
```
for word, count in word_count.items(): # البحث عن أعلى تكرار
```

```
    if count > max_count:
```

```
        max_count = count
```

```
most_common_word = word
```

```
print(f"({max_count} مرات) '{most_common_word}': الكلمة الأكثر تكراراً")
```



الموضوع السابع | مقارنة while vs for

نفس النتيجة بطريقتين مختلفتين

باستخدام while

i = 0

while i < 5:

print(i)

i += 1

باستخدام for

for i in range(5):

print(i)

الجانب	while	for
الاستخدام	عندما لا نعرف عدد التكرارات	عندما نعرف عدد التكرارات أو لدينا مجموعة
التحكم	يحتاج تحديث يدوي لمتغير التحكم	تتولى الحلقة التحديث تلقائياً
خطر الحلقة اللانهائية	عالي إذا نسيت التحديث	منخفض
البساطة	أكثر تعقيداً للمبتدئين	أبسط وأوضح

الموضوع السابع | مثال متقدم - لعبة تخمين الرقم



```
import random

# توليد رقم عشوائي بين 1 و 100
secret_number = random.randint(1, 100)
attempts = 0
max_attempts = 7

print("مرحباً بك في لعبة تخمين الرقم")
print(f"محاولات {max_attempts} لديك. 100 و 1 خمن رقماً بين")

while attempts < max_attempts:
    # استمر حتى انتهاء المحاولات
    try:
        # طلب تخمين من المستخدم
        guess = int(input(f"أدخل تخمينك: {attempts + 1} المحاولة"))
        attempts += 1 # زيادة عدد المحاولات

        if guess == secret_number:
            # إذا كان التخمين صحيح
            print(f"محاولات {attempts} في {secret_number} لقد خمنت الرقم!مبروك")
            break # خروج من الحلقة
        elif guess < secret_number:
            # إذا كان التخمين أصغر
            print("الرقم أكبر من تخمينك.")
        else:
            # إذا كان التخمين أكبر
            print("الرقم أصغر من تخمينك.")

        # إظهار المحاولات المتبقية
        remaining = max_attempts - attempts
        if remaining > 0:
            print(f"المحاولات المتبقية: {remaining}")

    except ValueError:
        # في حالة إدخال غير صحيح
        print("يرجى إدخال رقم صحيح")
        attempts -= 1 # لا نحسب هذه كمحاولة

    # إذا انتهت المحاولات دون تخمين صحيح
    if attempts >= max_attempts and guess != secret_number:
        print(f"الرقم {secret_number} الصحيح كان !انتهت المحاولات")
```

الموضوع السابع | أخطاء شائعة في الحلقات

١. نسيان تحديث متغير التحكم: 

خطأ شائع #

```
x = 0
while x < 5:
    print(x)
    x += 1 # نسيان
```

٣. تعديل القائمة أثناء التكرار: 

تجنب هذا

```
numbers = [1, 2, 3, 4, 5]
for num in numbers:
    if num % 2 == 0:
        numbers.remove(num) #
خطراً! تعديل القائمة أثناء التكرار
```

٢. استخدام الفهرس خارج النطاق: 

خطأ محتمل #

```
numbers = [1, 2, 3, 4, 5]
i = 0
while i <= len(numbers): # يجب أن يكون خطأ
    print(numbers[i]) # قد يتسبب في خطأ
    i += 1
```

الموضوع السابع | نصائح لاستخدام الحلقات بكفاءة

١. اختر النوع المناسب:

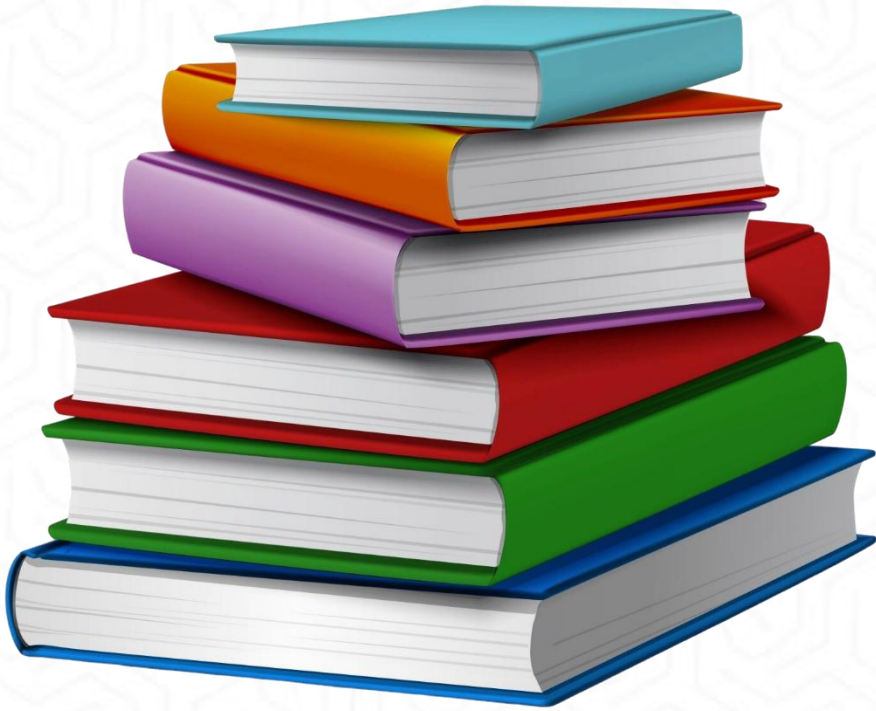
استخدم `for` للتكرار على المجموعات 

استخدم `while` للشروط المعقدة 

٢. تجنب الحلقات اللانهائية:

تأكد من تحديث متغير التحكم 

اختبر الشرط بعناية 



- Python for Everybody: Exploring Data in Python 3 Book by Charles Severance.



شكراً لكم



الكلية التطبيقية
الجامعة السعودية الإلكترونية
SAUDI ELECTRONIC UNIVERSITY
2011-1432

دبلوم الذكاء الاصطناعي التوليدي

اسم المقرر: البرمجة باستخدام بايثون

رمز المقرر: PYT103

الموضوع: جمل الشرط (if, elif, else)





الموضوع الخامس: جمل الشرط ((if, elif, else))

الأهداف

في نهاية هذا الدرس سيكون الطالب قادرًا على أن:

✓ فهم مفهوم التنفيذ الشرطي في البرمجة.

✓ كتابة جمل if البسيطة.

✓ استخدام العوامل المنطقية والمقارنة.

✓ تطبيق جمل elif للشروط المتعددة.

✓ بناء جمل شرطية متداخلة.

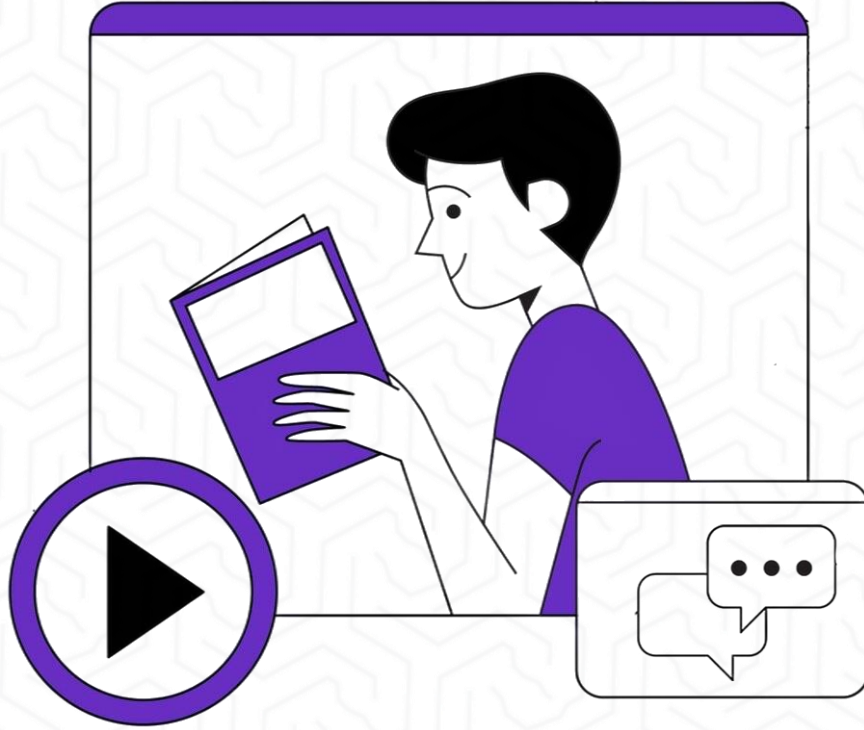
✓ استخدام التعبيرات المنطقية المعقدة.

✓ التعامل مع الاستثناءات باستخدام try/except



وصف الدرس

في هذا الدرس سوف نتناول :



يهدف هذا الدرس إلى تعليم الطلاب مفهوم التنفيذ الشرطي في لغة البرمجة Python، والذي يُعد أحد الأسس الجوهرية في البرمجة للتحكم في مسار تنفيذ البرنامج. سيتعلم الطلاب كيفية استخدام التعابير المنطقية وعوامل المقارنة لبناء شروط منطقية، ثم تطبيق هذه الشروط من خلال جمل `if` البسيطة للتحكم في تنفيذ مجموعة معينة من التعليمات. كما سيتقنون استخدام جملة `else` لتوفير بديل يتم تنفيذه عند عدم تحقق الشرط الأساسي، والانتقال إلى الشروط المتعددة باستخدام `elif` للتعامل مع الحالات المختلفة بطريقة منظمة ومرتبطة. سيغطي الدرس أيضاً موضوع الشروط المتداخلة وكيفية تبسيطها باستخدام العوامل المنطقية، بالإضافة إلى التعامل الآمن مع الاستثناءات والأخطاء المحتملة باستخدام بنية `try/except` ونمط الحماية (Guardian Pattern) خلال الدرس، سيتم تقديم أمثلة عملية متدرجة من البسيط إلى المعقد تشمل حساب الدرجات، أنظمة الأمان، الآلة الحاسبة، وإدارة المكتبات، مع التركيز على أفضل الممارسات في كتابة الشيفرة وتجنب الأخطاء الشائعة مثل الخلط بين عاملي الإسناد والمقارنة، وأهمية المسافات البادئة الصحيحة، وكيفية التعامل مع مقارنة النصوص بطريقة صحيحة. في نهاية الدرس، سيكون الطلاب قادرين على بناء برامج تفاعلية تتخذ قرارات ذكية بناءً على مدخلات المستخدم والظروف المختلفة، مما يمهّد الطريق لفهم مفاهيم برمجية أكثر تقدماً.



مقدمة - لماذا نحتاج للشروط؟

التحكم في تدفق البرنامج:

البرامج تحتاج لاتخاذ قرارات.

تنفيذ تعليمات مختلفة حسب الظروف

مثال من الحياة اليومية:

إذا كان الطقس ممطراً:

خذ معك مظلة

وإلا:

لا تحتاج لمظلة



Boolean Expressions التعبيرات المنطقية

التعبير المنطقي:

تعبير قيمته إما **True** أو **False**

يستخدم عوامل المقارنة:

```
x = 5
y = 10
print(x == y) # False
print(x < y)  # True
print(x != y) # True
```



عوامل المقارنة

عوامل المقارنة في Python

المعنى	العامل
يساوي	==
لا يساوي	!=
أكبر من	>
أصغر من	<
أكبر من أو يساوي	>=
أصغر من أو يساوي	<=

العوامل المنطقية

العوامل المنطقية الثلاثة

• **and:**

صحيح إذا كان كلا الطرفين صحيحاً

• **or:**

صحيح إذا كان أحد الطرفين صحيحاً

• **not:**

عكس القيمة المنطقية

```
x = 5  
print(x > 0 and x < 10) # True  
print(x < 0 or x > 3)  # True  
print(not x == 5)      # False
```

بنية جملة if الأساسية

• الصيغة العامة:

شرط: if
تعليمات تنفذ إذا كان الشرط صحيحاً

• مثال عملي:

```
x = 10  
if x > 0:  
    print('x is positive')
```

أهمية المسافات البادئة Indentation

قواعد مهمة:

- استخدم 4 مسافات للمسافة البادئة.
- جميع التعليمات في نفس الكتلة تحتاج نفس المسافة البادئة.
- انتهي السطر الأول ب ":".

```
x = 15
if x > 10:
    print("x is larger than 10")
    print("This line is within the if condition")
    print("This Line is out of if condition")
```

التنفيذ البديل - else

• عندما نريد تنفيذ شيء آخر:

```
الشرط: if
تعليمات إذا كان الشرط صحيحاً
else:
تعليمات إذا كان الشرط خاطئاً
```

• مثال عملي:

```
x = 7
if x % 2 == 0:
    print('x is even')
else:
    print('x is odd')
```



مثال تطبيقي - فحص العمر

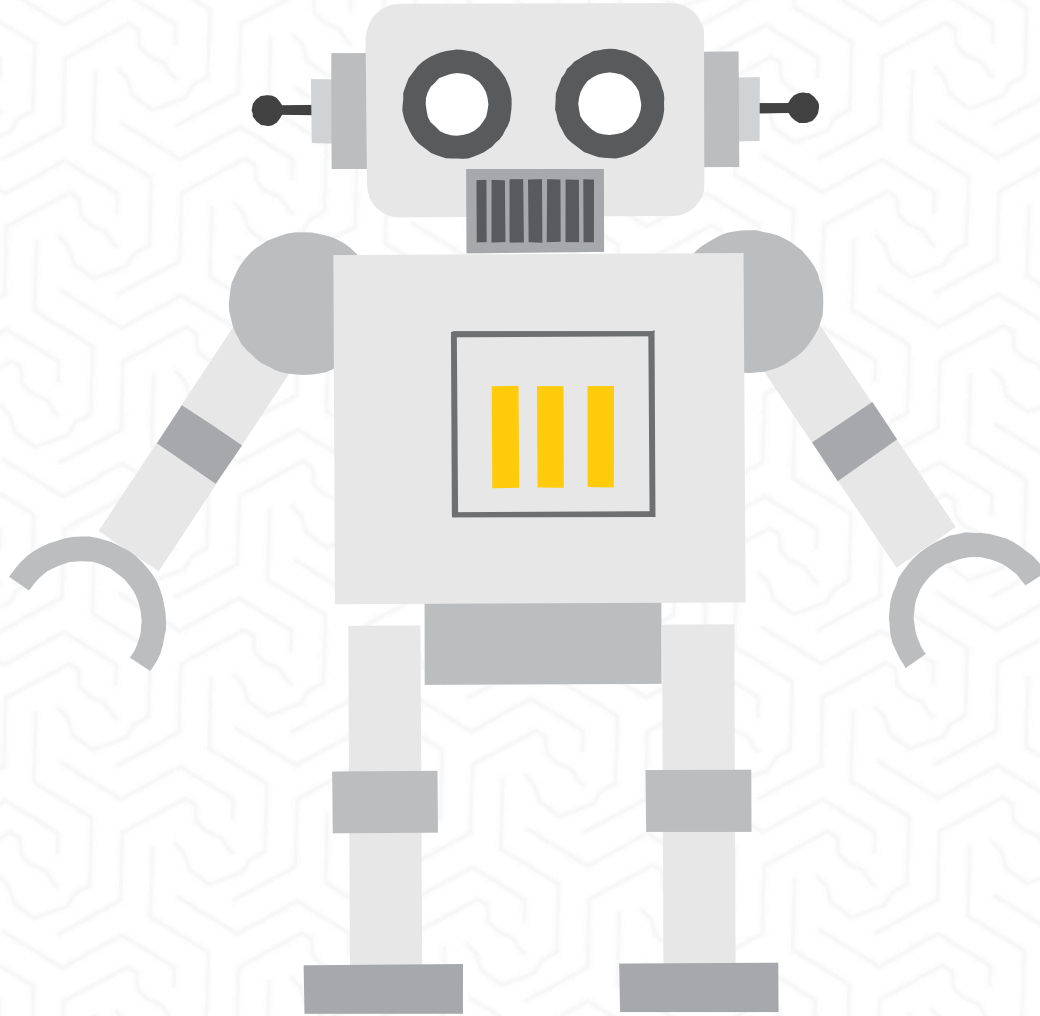
```
age = int(input('Enter your age: '))

if age >= 18:
    print('You can vote ')
    print('You are an adult')
else:
    print('You can not vote yet!')
print('Wait until you reach 18 years old')
```

اكتب برنامج يأخذ العمر من المستخدم ويقوم
بفحص العمر إذا كان 18 أو أكبر يطبع عبارة
بإمكانك التصويت لأنك بالغ
أما إذا كان العمر أقل يطبع عبارة لا تستطيع الصوت
انتظر حتى تبلغ 18 سنة

الشروط المتسلسلة - elif

عندما نحتاج لأكثر من خيارين:



1: شرط `if`

1 تعليمات

2: شرط `elif`

2 تعليمات

3: شرط `elif`

3 تعليمات

`else:`

تعليمات افتراضية

مثال - elif تقدير الدرجات

```
score = float(input('Enter your score : '))
```

```
    if score >= 90:  
        grade = 'A'  
    elif score >= 80:  
        grade = 'B'  
    elif score >= 70:  
        grade = 'C'  
    elif score >= 60:  
        grade = 'D'  
    else:  
        grade = 'F'
```

```
print(f'your grade is: {grade}')
```

الشروط المتداخلة

• شرط داخل شرط آخر:

```
x = 10
y = 20

if x > 0:
    print('x is even')
    if y > 0:
        print('y is also even')
        if x + y > 25:
            print('x and y sum is larger than 25')
        else:
            print('y is odd')
    else:
        print('x is odd')
```

عنوان | مثال الشروط المتداخلة - نظام الأمان

```
username = input('Enter username: ')
password = input(' Enter password: ')

    if username == 'admin':
        if password == '123456':
            print('Hello Administrator!')
            print('Logged in successfully ')
        else:
            print('Wrong provided password')
    else:
        print(' Username is not correct')
```

تبسيط الشروط المتداخلة

- باستخدام العوامل المنطقية نستطيع تبسيط الشروط المتداخلة:

```
# بدلاً من:  
if x > 0:  
    if x < 10:  
        print('x is a positive number between 0 and 10')
```

```
# يمكن كتابة:  
if 0 < x < 10:  
    print('x is a positive number between 0 and 10')
```

```
# أو:  
if x > 0 and x < 10:  
    print ('x is a positive number between 0 and 10')
```

التعامل مع الاستثناءات - try/except

حماية البرنامج من الأخطاء:

```
try:
    number = int(input('Enter an integer number: '))
    result = 10 / number
    print(f' The result: {result}')
except ValueError:
    print('Please provide an integer number ')
except ZeroDivisionError:
    print('You cannot divide on zero')
```

مثال شامل - آلة حاسبة بسيطة

```
try:
    num1 = float(input('Enter First number:'))
    operator = input('Provide Operation (+, -, *, /): ')
    num2 = float(input('Enter Second number: '))

    if operator == '+':
        result = num1 + num2
    elif operator == '-':
        result = num1 - num2
    elif operator == '*':
        result = num1 * num2
    elif operator == '/':
        if num2 != 0:
            result = num1 / num2
        else:
            print('Wrong: You divided the number by zero')
            exit()
    else:
        print('You provided wrong operation ')
        exit()

    print(f'The result: {result}')
except ValueError:
    print('Wrong: Please provide integer numbers')
```



نمط الحماية

• حماية من الأخطاء المحتملة:

```
x = 6
```

```
y = 0
```

نمط يحتمل الخطأ #

```
if x/y > 2:
```

```
print('result larger than 2')
```

نمط آمن #

```
if y != 0 and x/y > 2:
```

```
print('result larger than 2')
```



تجاوز التحقق

كيف تعمل العوامل المنطقية:

#في and: إذا كان الشرط الأول False،

#لن يتم فحص الشرط الثاني

x = 1

y = 0

if x >= 2 and (x/y) > 2: # آمن

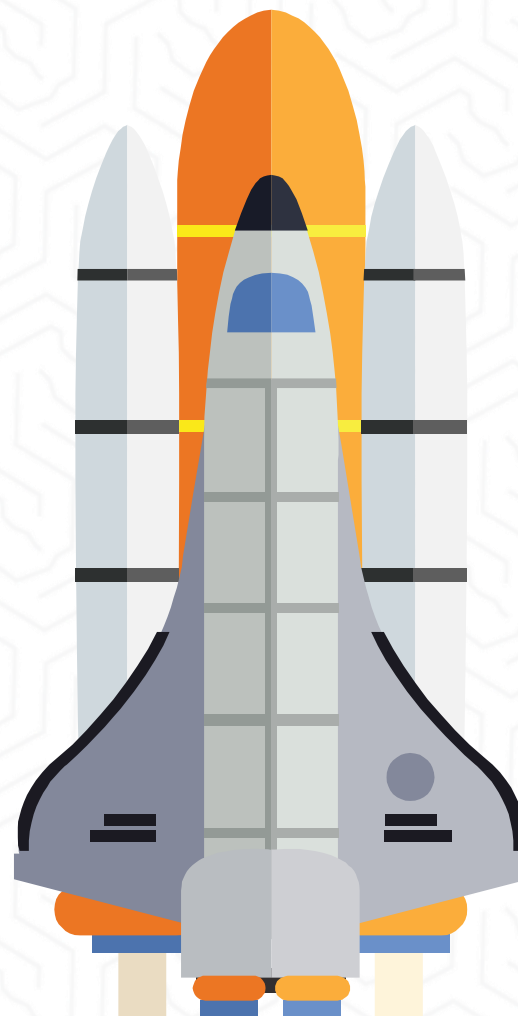
print('first result')

#في or: إذا كان الشرط الأول True،

#لن يتم فحص الشرط الثاني

if x >= 1 or (x/y) > 2: # آمن

print('second result')



Student Data Validation Program

مقارنة النصوص

أمور مهمة عند مقارنة النصوص:

```
name = input('أدخل اسمك: ')
```

مشكلة الأحرف الكبيرة والصغيرة

```
if name == 'ahmed':  
    print('مرحباً أحمد')
```

حل أفضل

```
if name.lower() == 'ahmed':  
    print('مرحباً أحمد')
```

فحص متعدد

```
if name.lower() in ['ahmed', 'أحمد', 'Ahmad']:  
    print('مرحباً أحمد')
```



الشروط مع القوائم والنصوص

```
# Check if element exists in a list
fruits = ['apple', 'orange', 'banana']
fruit = input('Enter a fruit name: ')
```

```
    if fruit in fruits:
        print(f'{fruit} is available')
    else:
        print(f'{fruit} is not available')
```

```
# Check for empty text
text = input('Enter some text: ')
if text: # True if not empty
    print(f'The text is: {text}')
else:
    print('You did not enter any text')
```



مثال متقدم - نظام إدارة المكتبة

Library Management System

```
books = {
    'One Hundred Years of Solitude': {'available': False, 'author': 'Gabriel Garcia Marquez'},
    'Les Misérables': {'available': True, 'author': 'Victor Hugo'}
}

print('Welcome to the Library System')
book_name = input('Enter book name: ')

if book_name in books:
    book_info = books[book_name]
    print(f'Book: {book_name}')
    print(f'Author: {book_info["author"]}')

    if book_info['available']:
        action = input('Book is available. Do you want to borrow it? (yes/no): ')
        if action.lower() in ['yes', 'y']:
            books[book_name]['available'] = False
            print('Book borrowed successfully!')
        else:
            print('Thank you')
        else:
            print('Book is currently not available')
        else:
            print('Book not found in the library')

print('Program ended.')
```



أخطاء شائعة في الشروط

أخطاء يجب تجنبها:

1- استخدام = بدلاً من ==.

2- نسيان النقطتين (:).

3- المسافات البادئة الخاطئة.

4- مقارنة النصوص دون

مراعاة الحالة.

خطأ #
if x = 5:
print('x يساوي 5')

صحيح #
if x == 5:
print('x يساوي 5')

خطأ #
if x > 5
print('x أكبر من 5')

صحيح #
if x > 5:
print('x أكبر من 5')

خطأ #
if x > 5:
print('x أكبر من 5')
لا توجد
مسافة بادئة

صحيح #
if x > 5:
print('x أكبر من 5')

قد لا يعمل كما متوقع #
name = 'Ahmed'
if name == 'ahmed':
print('مرحباً')

أفضل #
if name.lower() == 'ahmed':
print('مرحباً')

نصائح للكتابة الجيدة

1- تجنب الشروط المعقدة جداً.

معقد
if (x > 5 and y < 10) or (z == 0 and w != 5) or (a > b and c < d):
 print('شروط معقد')

قسم إلى خطوات - أفضل
condition1 = x > 5 and y < 10
condition2 = z == 0 and w != 5
condition3 = a > b and c < d

if condition1 or condition2 or condition3:
 print('شروط واضح')





المراجع

Python for Everybody:
Exploring Data in
Book by Python 3
Charles Severance.



شكراً لكم





الكلية التطبيقية

الجامعة السعودية الإلكترونية

SAUDI ELECTRONIC UNIVERSITY

2011-1432

دبلوم الذكاء الاصطناعي التوليدي

اسم المقرر: : البرمجة باستخدام بايثون

رمز المقرر: PYT103

الموضوع: الرابع



الموضوع الرابع:

هياكل البيانات في بايثون - القوائم والصفوف

الجامعة الوطنية

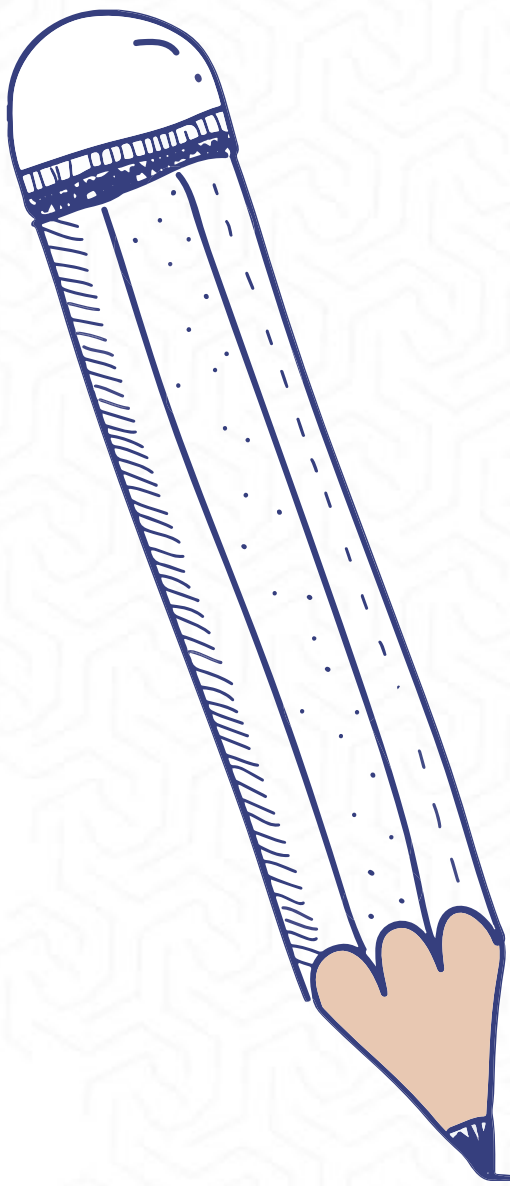
في نهاية هذا الدرس ستكون قادرًا على أن:

- ✓ تفهم مفهوم القوائم والصفوف كهيكل بيانات أساسية.
- ✓ تنشئ ومعالجة القوائم باستخدام العمليات المختلفة.
- ✓ تستخدم التوابع المختلفة للقوائم والصفوف.
- ✓ تميز بين القوائم والصفوف ومعرفة متى نستخدم كل منهما.
- ✓ تطبق مفاهيم القوائم والصفوف في حل المشاكل البرمجية الحقيقية.
- ✓ تفهم التسمية البديلة وتأثيرها على البرمجة.



الموضوع الرابع | وصف الدرس

- هذا الدرس يغطي هياكل البيانات الأساسية في بايثون:
- القوائم Lists: هياكل بيانات مرنة وقابلة للتعديل.
- الصفوف Tuples: هياكل بيانات ثابتة وغير قابلة للتعديل.
- العمليات والتوابع: المتاحة لكل نوع من هياكل البيانات.
- التطبيقات العملية: استخدام هذه الهياكل في البرمجة الحقيقية.



تعريف القائمة: List

١. القائمة هي سلسلة من القيم، حيث:
٢. يمكن أن تحتوي على قيم من أي نوع بيانات.
٣. القيم تُفهرس باستخدام الأعداد الصحيحة.
٤. قابلة للتعديل Mutable.

إنشاء قوائم مختلفة

```
numbers = [10, 20, 30, 40]
fruits = ['apple', 'banana', 'orange']
mixed = ['spam', 2.0, 5, [10, 20]]
empty = []
```

يتم انشاء القائمة باستخدام []

```
# الطريقة المباشرة  
cheeses = ['Cheddar', 'Edam', 'Gouda']
```

```
# قائمة فارغة  
empty_list = []
```

```
# استخدام list()  
numbers = list()
```

```
# تحويل سلسلة نصية إلى قائمة  
word = 'spam'  
letters = list(word)  
print(letters) # ['s', 'p', 'a', 'm']
```

استخدام الفهارس Indexing

```
fruits = ['apple', 'banana', 'orange', 'grape']
```

الوصول للعناصر

```
print(fruits[0]) # 'apple'
```

```
print(fruits[2]) # 'orange'
```

```
print(fruits[-1]) # 'grape' (العنصر الأخير)
```

```
print(fruits[-2]) # 'orange' (ما قبل الأخير)
```

طول القائمة

```
print(len(fruits)) # 4
```

تعديل عناصر القائمة

```
numbers = [17, 123, 45]  
print(numbers)    # [17, 123, 45]
```

```
    تعديل عنصر #  
numbers[1] = 5  
print(numbers)    # [17, 5, 45]
```

هذا مختلف عن السلاسل النصية التي لا يمكن تعديلها #

```
string = "hello"  
string[0] = "H"  خطأ! #
```

استخدام حلقة for

```
cheeses = ['Cheddar', 'Edam', 'Gouda']
```

```
# المرور على العناصر مباشرة  
for cheese in cheeses:  
    print(cheese)
```

```
# المرور باستخدام الفهارس  
for i in range(len(cheeses)):  
    print(f"Index {i}: {cheeses[i]}")
```

```
# تعديل العناصر  
numbers = [1, 2, 3, 4]  
for i in range(len(numbers)):  
    numbers[i] = numbers[i] * 2  
print(numbers) # [2, 4, 6, 8]
```

الجمع والضرب

جمع القوائم

a = [1, 2, 3]

b = [4, 5, 6]

c = a + b

print(c) # [1, 2, 3, 4, 5, 6]

ضرب القوائم (التكرار)

zeros = [0] * 4

print(zeros) # [0, 0, 0, 0]

repeat = [1, 2, 3] * 3

print(repeat) # [1, 2, 3, 1, 2, 3, 1, 2, 3]

الحصول على أجزاء من القائمة

```
t = ['a', 'b', 'c', 'd', 'e', 'f']

# تجزئة أساسية
print(t[1:3]) # ['b', 'c']
print(t[:4])  # ['a', 'b', 'c', 'd']
print(t[3:])  # ['d', 'e', 'f']
print(t[:])   # ['a', 'b', 'c', 'd', 'e', 'f'] نسخة كاملة

# تعديل عدة عناصر
t[1:3] = ['x', 'y']
print(t) # ['a', 'x', 'y', 'd', 'e', 'f']
```

extend و append

```
# إضافة عنصر واحد
fruits = ['apple', 'banana']
fruits.append('orange')
print(fruits) # ['apple', 'banana', 'orange']
```

```
# extend: إضافة عناصر متعددة
fruits.extend(['grape', 'mango'])
print(fruits) # ['apple', 'banana', 'orange', 'grape', 'mango']
```

```
# الفرق بين append و extend
list1 = [1, 2, 3]
list1.append([4, 5])
print(list1) # [1, 2, 3, [4, 5]]
```

```
list2 = [1, 2, 3]
list2.extend([4, 5])
print(list2) # [1, 2, 3, 4, 5]
```

sorted و sort

```
# ترتيب القائمة نفسها
numbers = [3, 1, 4, 1, 5, 9, 2, 6]
numbers.sort()
print(numbers) # [1, 1, 2, 3, 4, 5, 6, 9]

# ترتيب نازل
numbers.sort(reverse=True)
print(numbers) # [9, 6, 5, 4, 3, 2, 1, 1]

# إنشاء قائمة جديدة مرتبة
original = [3, 1, 4, 1, 5]
sorted_list = sorted(original)
print(original) # [3, 1, 4, 1, 5] لم تتغير
print(sorted_list) # [1, 1, 3, 4, 5]
```

طرق مختلفة لحذف العناصر

```
# حذف وإرجاع عنصر: pop
numbers = [1, 2, 3, 4, 5]
x = numbers.pop(1) # حذف العنصر في الفهرس 1
print(x)           # 2
print(numbers)     # [1, 3, 4, 5]
```

```
# بدون فهرس: حذف العنصر الأخير pop()
last = numbers.pop()
print(last)        # 5
print(numbers)     # [1, 3, 4]
```

```
# حذف بالقيمة: remove
numbers.remove(3)
print(numbers)     # [1, 4]
```

```
# حذف بالفهرس: del
del numbers[0]
print(numbers)     # [4]
```

max, min, sum, len

```
nums = [3, 41, 12, 9, 74, 15]

print(len(nums))      # عدد العناصر - 6
print(max(nums))      # أكبر قيمة - 74
print(min(nums))      # أصغر قيمة - 3
print(sum(nums))      # مجموع القيم - 154
```

حساب المتوسط

```
average = sum(nums) / len(nums)
print(average)        # 25.666...
```

مع السلاسل النصية

```
words = ['python', 'java', 'c++']
print(max(words))     # ترتيب أبجدي 'python'
print(min(words))     # 'c++'
```

التحويل بين القوائم والنصوص

```
# تحويل نص إلى قائمة
text = 'pining for the fjords'
words = text.split()
print(words) # ['pining', 'for', 'the', 'fjords']
```

```
# تحويل نص إلى أحرف
word = 'spam'
letters = list(word)
print(letters) # ['s', 'p', 'a', 'm']
```

```
# تجميع قائمة إلى نص
words = ['pining', 'for', 'the', 'fjords']
result = ' '.join(words)
print(result) # 'pining for the fjords'
```

```
# استخدام محدد مختلف
result2 = '-'.join(words)
print(result2) # 'pining-for-the-fjords'
```

البحث في القوائم

```
fruits = ['apple', 'banana', 'orange']
```

فحص وجود عنصر

```
print('banana' in fruits)      # True
print('grape' in fruits)      # False
print('apple' not in fruits)  # False
```

استخدام في الشروط

```
if 'banana' in fruits:
    print("We have bananas!")
```

البحث في قائمة من الأرقام

```
numbers = [1, 2, 3, 4, 5]
if 3 in numbers:
    print("Found number 3")
```

فهم كيفية عمل القوائم في الذاكرة

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b)  # True نفس القيم
print(a is b)  # False كائنات مختلفة
```

```
# المراجع
a = [1, 2, 3]
b = a  # b يشير لنفس القائمة
print(a is b)  # True
```

```
# تعديل أحدهما يؤثر على الآخر
b[0] = 100
print(a)  # [100, 2, 3]
print(b)  # [100, 2, 3]
```

المشاكل والحلول

مشكلة التسمية البديلة

```
original = [1, 2, 3, 4]
alias = original
alias.append(5)
print(original) # [1, 2, 3, 4, 5] تغيرت!
```

الحل: إنشاء نسخة

```
original = [1, 2, 3, 4]
copy1 = original[:] # تجزئة كاملة
copy2 = list(original) # استخدام list()
copy3 = original.copy() # استخدام copy()

copy1.append(5)
print(original) # [1, 2, 3, 4] لم تتغير
print(copy1) # [1, 2, 3, 4, 5]
```

تمرير القوائم للتوابع

```
def delete_head(lst):  
    """ حذف العنصر الأول من القائمة """  
    if len(lst) > 0:  
        del lst[0]  
  
def get_tail(lst):  
    """ إرجاع قائمة جديدة بدون العنصر الأول """  
    if len(lst) <= 1:  
        return []  
    return lst[1:]  
  
# الاستخدام  
numbers = [1, 2, 3, 4, 5]  
delete_head(numbers)  
print(numbers) # [2, 3, 4, 5] تم تعديل القائمة الأصلية  
  
numbers = [1, 2, 3, 4, 5]  
tail = get_tail(numbers)  
print(numbers) # [1, 2, 3, 4, 5] لم تتغير  
print(tail)    # [2, 3, 4, 5]
```

برنامج لحساب إحصائيات قائمة أرقام

```
def calculate_stats(numbers):  
    """ حساب إحصائيات قائمة من الأرقام """  
    if not numbers:  
        return None  
  
    stats = {  
        'count': len(numbers),  
        'sum': sum(numbers),  
        'average': sum(numbers) / len(numbers),  
        'min': min(numbers),  
        'max': max(numbers)  
    }  
    return stats  
  
# الاستخدام  
grades = [85, 92, 78, 96, 88, 79, 91]  
results = calculate_stats(grades)  
  
for key, value in results.items():  
    print(f"{key}: {value:.2f}")
```

ما هي الصفوف؟

الصف Tuple هو سلسلة من القيم مثل القائمة، لكن:

- غير قابل للتعديل Immutable
- قابل للمقارنة والترتيب
- يمكن استخدامه كمفتاح في القواميس

```
# إنشاء صف
point = (3, 4)
coordinates = (10.5, 20.3, 15.7)
mixed = ('hello', 42, 3.14, True)

# بدون أقواس (لكن الأقواس أوضح)
t = 'a', 'b', 'c'
print(type(t)) # <class 'tuple'>
```

طرق مختلفة لإنشاء الصفوف

```
# صف فارغ
empty_tuple = ()
empty_tuple2 = tuple()

# (ملاحظة الفاصلة) صف بعنصر واحد
single = (42,) # أو single = 42,
print(type(single)) # <class 'tuple'>

# بدون فاصلة سيكون النوع مختلف
not_tuple = (42)
print(type(not_tuple)) # <class 'int'>

# تحويل قائمة إلى صف
list_data = [1, 2, 3, 4]
tuple_data = tuple(list_data)
print(tuple_data) # (1, 2, 3, 4)
```

الفهرسة والتجزئة

```
colors = ('red', 'green', 'blue', 'yellow')
```

#الوصول بالفهرس

```
print(colors[0]) # 'red'  
print(colors[-1]) # 'yellow'  
print(len(colors)) # 4
```

#التجزئة

```
print(colors[1:3]) # ('green', 'blue')  
print(colors[:2]) # ('red', 'green')  
print(colors[2:]) # ('blue', 'yellow')
```

#لا يمكن التعديل

```
colors[0] = 'purple' # TypeError! #
```

العمليات المتاحة

```
# الجمع
tuple1 = (1, 2, 3)
tuple2 = (4, 5, 6)
combined = tuple1 + tuple2
print(combined) # (1, 2, 3, 4, 5, 6)

# الضرب (التكرار)
repeated = (1, 2) * 3
print(repeated) # (1, 2, 1, 2, 1, 2)

# العضوية
print(2 in tuple1) # True
print(10 in tuple1) # False

# التوابع الجاهزة
numbers = (3, 1, 4, 1, 5, 9)
print(len(numbers)) # 6
print(max(numbers)) # 9
print(min(numbers)) # 1
print(sum(numbers)) # 23
```

الترتيب والمقارنة

```
# المقارنة عنصر بعنصر
print((0, 1, 2) < (0, 3, 4))      # True
print((0, 1, 2000000) < (0, 3, 4)) # True
```

```
#ترتيب قائمة من الصفوف
points = [(3, 4), (1, 2), (5, 1), (2, 8)]
points.sort()
print(points) # [(1, 2), (2, 8), (3, 4), (5, 1)]
```

```
#ترتيب حسب العنصر الثاني
points.sort(key=lambda x: x[1])
print(points) # [(5, 1), (1, 2), (3, 4), (2, 8)]
```

ترتيب متقدم باستخدام الصفوف

```
# ترتيب الكلمات حسب الطول
text = 'but soft what light in yonder window breaks'
words = text.split()
# Decorate: إضافة معيار الترتيب
decorated = []
for word in words:
    decorated.append((len(word), word))
# Sort: ترتيب الصفوف
decorated.sort(reverse=True)
# Undecorate: استخراج الكلمات
result = []
for length, word in decorated:
    result.append(word)
print(result)

# ['yonder', 'window', 'breaks', 'light', 'soft', 'what', 'but', 'in']
```

تقسيم الصفوف

```
# إسناد متعدد
point = (3, 4)
x, y = point
print(f"x = {x}, y = {y}") # x = 3, y = 4

# مع أكثر من متغيرين
person = ('Alice', 25, 'Engineer')
name, age, job = person
print(f"{name} is {age} years old and works as {job}")

# تبديل قيم متغيرين
a = 10
b = 20
a, b = b, a # تبديل بدون متغير مؤقت
print(f"a = {a}, b = {b}") # a = 20, b = 10

# استخدام في الحلقات
pairs = [(1, 'one'), (2, 'two'), (3, 'three')]
for number, word in pairs:
    print(f"{number}: {word}")
```

استخدام الصفوف كمفاتيح

```
# الصفوف يمكن أن تكون مفاتيح في القواميس
# لأنها غير قابلة للتعديل
coordinates = {}
coordinates[(0, 0)] = 'origin'
coordinates[(1, 1)] = 'point A'
coordinates[(-1, 2)] = 'point B'
print(coordinates)
print(coordinates[(0, 0)]) # 'origin'
# مثال: تخزين إحداثيات الشطرنج
chess_board = {}
chess_board[('a', 1)] = 'white rook'
chess_board[('e', 1)] = 'white king'
chess_board[('a', 8)] = 'black rook'
# القوائم لا يمكن أن تكون مفاتيح!
#coordinates[[0, 0]] = 'test' # TypeError!
```

الموضوع الرابع | متى نستخدم القوائم ومتى نستخدم الصفوف؟

القواعد العامة للاختيار

الصفوف Tuples	القوائم Lists
البيانات ثابتة	البيانات قابلة للتغيير
عدد العناصر ثابت	عدد العناصر متغير
أنواع بيانات مختلفة	نفس نوع البيانات
مثل: إحداثيات نقطة	مثل: قائمة أسماء

```
# استخدام القوائم
shopping_list = ['milk', 'bread', 'eggs']
shopping_list.append('cheese') # يمكن إضافة عناصر

# استخدام الصفوف
point = (3, 4) # نقطة ثابتة
rgb_color = (255, 128, 0) # لون ثابت
student_record = ('Ahmed', 'Ali', 123456, 'CS') # سجل طالب
```